HOG_LBP_KAZE_SVM_RF_Kas_Kod_Metodoloji_Raporu.pdf - local repository text copy

Kaş Bölgesi Deepfake Tespiti - Kod ve Metodoloji Raporu
HOG + LBP + KAZE ile SVM ve Random
Forest Tabanlı
Kaş Bölgesi Deepfake Tespiti
Kod ve Metodoloji Raporu
İncelenen notebook:
HOG_LBP_KAZE_SVM_RF_Kas_Deepfake_Colab_OPENCV4_FIXED (1).ipynb
Çalışma bölgesi: Kaş (Eyebrow ROI)
Sınıflandırma: Real / Fake
Yöntemler: HOG, LBP, KAZE, RBF-SVM, Random Forest
Kaş Bölgesi Deepfake Tespiti - Kod ve Metodoloji Raporu
1. Raporun Amacı ve Kapsamı
Bu rapor, verilen Google Colab notebookunun ne yaptığını, hangi veri akışını kullandığını, özelliklerin nasıl çıkarıldığını, modellerin nasıl eğitildiğini, model seçiminin hangi kurala göre yapıldığını ve deney çıktılarının nasıl kaydedildiğini ayrıntılı olarak açıklamaktadır. Rapor bir sonuç yorumundan ziyade kodun metodolojisini ve teknik işleyişini belgelemek amacıyla hazırlanmıştır.
Notebook, daha önce kaş bölgesine kırpılmış görüntüleri doğrudan kullanır; görüntüler üzerinde yeniden yüz tespiti veya kaş kırpma işlemi gerçekleştirmez. Hazır ROI görüntülerinden HOG, LBP ve KAZE özellikleri çıkarılır ve üç ayrı deney tasarımıyla SVM ve Random Forest sınıflandırıcıları değerlendirilir.
Temel deney mantığı: E1 HOG+LBP ile RBF-SVM; E2 HOG+LBP+KAZE ile RBF-SVM; E3 HOG+LBP+KAZE ile Random Forest kullanır.
2. Genel Sistem Akışı
Şekil 1. Notebookun uçtan uca veri ve modelleme akışı.
Kodun genel akışı, hazır kaş ROI görüntülerinin okunmasıyla başlar. Görüntüler standart boyuta getirilip gri seviyeye çevrilir. Ardından HOG, LBP ve KAZE özellikleri çıkarılır. Özellikler belirlenen deney tasarımına göre birleştirilir, train/validation/test ayrımı korunur, modeller train setinde eğitilir, hiperparametre seçimi validation seti üzerinde yapılır ve yalnızca seçilen model bağımsız test setinde değerlendirilir.
3. Girdi Veri Yapısı ve Etiketleme
```
Kaş/
├── real/
│   ├── train/
│   ├── val/
│   └── test/
├── fake/
│   ├── train/
│   ├── val/
│   └── test/
```
Kaş Bölgesi Deepfake Tespiti - Kod ve Metodoloji Raporu
```
└── metadata.csv
```
Kod, veri yapısını değiştirmeden mevcut klasörlerden görüntüleri tarar. Real sınıfı 0, fake sınıfı 1 olarak etiketlenir. JPG, JPEG, PNG, BMP ve WEBP uzantıları desteklenmektedir. Train, validation ve test klasörlerinin her birinde iki sınıfın da bulunması zorunlu tutulur; aksi durumda kod hata vererek deneyi durdurur.
4. Ortam Kurulumu ve KAZE Uyumluluğu
İlk kod hücresi deneyin çalışacağı yazılım ortamını hazırlamaktadır. Notebook KAZE özelliğini kullandığı için OpenCV sürüm uyumluluğunu özel olarak yönetir. Mevcut opencv-python, opencv-python-headless, opencv-contrib-python ve opencv-contrib-python-headless paketleri kaldırılır; ardından opencv-contrib-python-headless==4.13.0.92 sürümü kurulur.
☐ OpenCV contrib sürümü KAZE algoritmasına erişim için sabitlenir.
☐ scikit-image HOG ve LBP çıkarımı için kullanılır.
☐ scikit-learn SVM, Random Forest ve değerlendirme metriklerini sağlar.
☐ joblib eğitilmiş modelleri kalıcı olarak kaydetmek için kullanılır.
☐ PyYAML deney konfigürasyonunu YAML formatında saklamak için kullanılır.
☐ SEED=42 ile NumPy ve Python random işlemleri mümkün olduğunca tekrarlanabilir hale getirilir.
KAZE API'sinin farklı OpenCV Python binding'lerinde iki farklı isimle bulunabilmesi dikkate alınmıştır. Kod önce cv2.KAZE_create, ardından cv2.KAZE.create seçeneğini kontrol eder. KAZE nesnesi gerçekten oluşturulamazsa deney başlamadan hata üretilir. Bu kontrol, feature çıkarım aşamasında sessiz bir uyumsuzluk oluşmasını önler.
5. local source package ve Çıktı Yönetimi
İkinci kod hücresi local source package'ı bağlar ve deneyin sabit giriş/çıkış yollarını tanımlar. Girdi klasörü Nazlıcan/Deney 1/Kaş olarak belirlenmiştir. Sonuçlar, Sonuçlar/HOG_LBP_KAZE_SVM_RF_Kas_Deneyi_Sonuc_ klasörü altında standart bir deney paketi şeklinde tutulur.

Sonuçlar/HOG_LBP_KAZE_SVM_RF_Kas_Deneyi_Sonuc_/
├── checkpoints/
├── logs/
├── metrics/
├── predictions/
├── figures/
├── artifacts/
│ └── feature_chunks/
├── config_resolved.yaml
├── requirements_lock.txt
├── environment.json
├── run_summary.json
└── output_manifest.csv
Kaş Bölgesi Deepfake Tespiti - Kod ve Metodoloji Raporu
Ayrıca her çalıştırmaya zaman damgası içeren bir RUN_ID atanır ve training.log dosyasına temel süreç bilgileri yazılır. Böylece hangi veri yoluyla, hangi çalıştırmanın ne zaman üretildiği sonradan izlenebilir.
6. Veri Setinin Taranması ve Metadata Birleştirme
Kod real/fake ve train/val/test kombinasyonlarının tamamını gezer ve her görüntü için image_path, relative_path, file_name, class_name, label ve split alanlarını içeren bir pandas DataFrame oluşturur. Bu tablo deneyin temel veri indeksidir ve artifacts/dataset_index.csv dosyasına kaydedilir.
Kaş klasöründe metadata.csv bulunursa kod output_relative_path alanı üzerinden görüntü indeksini metadata ile birleştirir. Mevcut olduğu ölçüde sample_id, source_video, frame_index, face_index, status, output_sha256, roi_width, roi_height, blur_score ve eye_landmark_fraction alanları da deneye taşınır. Böylece yalnızca görüntü dosyaları değil, ROI üretim sürecinden gelen izlenebilirlik bilgileri de korunur.
7. Veri Sızıntısı (Leakage) Kontrolü
Notebook, train/validation/test ayrımının güvenilirliğini artırmak için iki denetim uygular.
1. Exact ROI hash audit: output_sha256 alanı varsa aynı görüntü içeriğinin birden fazla splitte bulunup bulunmadığı kontrol edilir.
2. Source-video audit: source_video bilgisi doluysa aynı kaynak videonun birden fazla splitte bulunup bulunmadığı kontrol edilir.
Herhangi bir hash veya kaynak-video sızıntısı saptanırsa eğitim başlamadan RuntimeError oluşturulur. source_video bilgisi mevcut değilse kod bunu PASSED olarak işaretlemek yerine NOT_AVAILABLE olarak raporlar. Bu yaklaşım, doğrulanmamış bir koşulun yanlış biçimde doğrulanmış gibi sunulmasını engeller.
8. Görüntü Ön İşleme
Her kaş ROI görüntüsü cv2.imread ile renkli olarak okunur. Ardından görüntü 224×224 piksele yeniden boyutlandırılır ve gri seviyeye dönüştürülür. Feature çıkarımı renk kanalları yerine yoğunluk ve lokal yapı bilgisi üzerinden yapılır.
IMAGE_SIZE = (224, 224)
image = cv2.resize(image, IMAGE_SIZE, interpolation=cv2.INTER_AREA)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
Kaş Bölgesi Deepfake Tespiti - Kod ve Metodoloji Raporu
9. HOG Özellik Çıkarımı
HOG (Histogram of Oriented Gradients), görüntüdeki lokal gradyan yönlerini histogramlar halinde temsil eder. Bu notebookta kaşın çizgisel yapısı, kılların yönelimleri, kaş sınırları ve yapay üretim kaynaklı kenar/şekil bozulmalarını sayısal bir özellik vektörüne dönüştürmek amacıyla kullanılır.

HOG Parametresi	Kodda Kullanılan Değer
orientations	9
pixels_per_cell	16 × 16
cells_per_block	2 × 2
block_norm	L2-Hys
transform_sqrt	True
çıktı boyutu	6084

10. LBP Özellik Çıkarımı
LBP (Local Binary Pattern), bir pikselin çevresindeki komşuluk ilişkilerini ikili örüntülere dönüştürerek lokal doku bilgisini temsil eder. Kaş ROI'sinde deri-kaş geçişleri, lokal doku düzeni ve mikro düzeyde yapay üretim izleri gibi bilgilerin yakalanması amaçlanır.

LBP Parametresi	Kodda Kullanılan Değer
radius	3
points	24
method	uniform
histogram bin sayısı	26
çıktı boyutu	26

LBP sonucu doğrudan tüm piksel değerleriyle kullanılmaz. 26 binlik normalize bir histogram oluşturulur ve bu histogram 26 boyutlu sabit uzunluklu bir feature vektörü olarak modele

verilir.
11. KAZE Özellik Çıkarımı
KAZE, doğrusal olmayan ölçek uzayında anahtar noktalar (keypoints) ve lokal descriptorlar üretir. Görüntüden görüntüye anahtar nokta sayısı değişebileceği için ham KAZE descriptorlarını doğrudan klasik bir sınıflandırıcıya vermek mümkün değildir; her örnek için feature uzunluğunun sabit olması gerekir.
Notebook bu problemi descriptorların sütun bazında ortalamasını ve standart sapmasını alarak çözer. KAZE descriptor boyutu 64 olduğundan 64 ortalama + 64 standart sapma birleştirilir ve her görüntü için 128 boyutlu sabit bir KAZE özelliği elde edilir. Hiç keypoint bulunamazsa 128 uzunluğunda sıfır vektörü döndürülür.
KAZE = mean(descriptors)[64] + std(descriptors)[64]
Toplam KAZE feature = 128
Kaş Bölgesi Deepfake Tespiti - Kod ve Metodoloji Raporu
12. Feature Fusion ve Boyutlar
Notebook iki farklı feature seti üretir. Birinci set HOG ve LBP'nin yatay birleştirilmesiyle oluşturulur. İkinci set HOG, LBP ve KAZE'nin tamamını içerir.
Feature Seti Bileşenler Toplam Boyut
X_hog_lbp 6084 HOG + 26 LBP 6110
X_fused 6084 HOG + 26 LBP + 128
KAZE
6238
Kod ilk görüntü üzerinde feature boyutlarını hesaplayıp assert ifadeleriyle 6084, 26, 128 ve 6238 değerlerini doğrular. Böylece feature fonksiyonlarında beklenmeyen bir parametre veya sürüm değişikliği olması durumunda deney sessizce farklı boyutta veri üretmez.
13. Chunk Tabanlı Feature Çıkarımı ve Resume Mekanizması
Feature çıkarımı tüm görüntüler üzerinde tek seferde yürütülmek yerine 250 örneklik parçalar (chunk) halinde gerçekleştirilir. Her parça artifacts/feature_chunks altında sıkıştırılmış NPZ dosyası olarak kaydedilir.
☐ CHUNK_SIZE = 250 olarak tanımlıdır.
☐ Her chunk için beklenen relative_path sırası kaydedilir.
☐ Aynı chunk daha önce hesaplandıysa dosya açılır ve relative_path dizisi mevcut veriyle karşılaştırılır.
☐ Yollar birebir eşleşiyorsa featurelar yeniden hesaplanmaz; cache kullanılır.
☐ Yollar değişmişse chunk yeniden üretilir.
☐ Tüm chunklar sonunda vstack/concatenate ile tek feature matrisi haline getirilir.
Bu tasarım özellikle Colab oturumunun kesilmesi durumunda zaman kazandırır. Feature çıkarımı tamamlandıktan sonra features_full.npz ve feature_metadata.csv dosyaları da oluşturulur.
14. Train / Validation / Test Ayrımı
Kod yeni bir random split üretmez. Klasör yapısında önceden tanımlanmış train, val ve test etiketlerini aynen korur. split_all dizisinden boolean maskeler üretilerek hem HOG+LBP hem de fused feature matrisleri ayrı ayrı train, validation ve test alt kümelerine ayrılır.
Bu yaklaşımın temel amacı deneyler arasında aynı veri ayrımını korumaktır. Validation seti hiperparametre ve model konfigürasyonu seçimi için, test seti ise seçim tamamlandıktan sonra nihai bağımsız değerlendirme için kullanılır.
Kaş Bölgesi Deepfake Tespiti - Kod ve Metodoloji Raporu
15. Değerlendirme Metrikleri
evaluate_binary fonksiyonu sınıflandırma performansını tek bir standart üzerinden hesaplar. Fake sınıfı pozitif sınıf olarak kabul edilir.
Metrik Kodda Hesaplanma Amacı Yorum
Accuracy Tüm doğru tahminlerin toplam örneklere oranı
Genel doğruluk
Precision (Fake) Fake denilen örneklerin ne kadarının gerçekten fake olduğu
Yanlış alarm hassasiyeti
Recall (Fake) Gerçek fake örneklerin ne kadarının yakalandığı
Deepfake yakalama oranı
F1 (Fake) Precision ve recall'un harmonik ortalaması
Dengeli fake performansı
ROC-AUC Skorların sınıfları ayırma kapasitesi
Eşik bağımsız ayrıştırma gücü
Specificity (Real) Gerçek real örneklerin doğru real tanınma oranı
Gerçek görüntüleri koruma

TN/FP/FN/TP Confusion matrix bileşenleri Hata tiplerinin ayrıntısı
16. Model Seçim Kuralı
best_validation_row fonksiyonu validation sonuçlarını önce validation_f1_fake, eşitlik halinde validation_roc_auc değerine göre azalan biçimde sıralar. Bu nedenle model seçiminin birincil hedefi fake sınıfındaki F1 skorudur. Test metrikleri seçim sürecine dahil edilmez.
Bilimsel avantaj: Test setinin hiperparametre seçimine karıştırılmaması, test performansının daha bağımsız bir genelleme ölçümü olarak kalmasını sağlar.
17. Deney E1 - HOG + LBP ile RBF-SVM
Birinci deneyde 6110 boyutlu HOG+LBP feature vektörü kullanılır. Featurelar StandardScaler ile standardize edildikten sonra RBF çekirdekli Support Vector Machine modeline verilir.
☐ C adayları: 0.1, 1.0 ve 10.0
☐ kernel: rbf
☐ gamma: scale
☐ class_weight: balanced
☐ probability: False
☐ Model her C değeri için train setinde eğitilir ve validation setinde değerlendirilir.
☐ Validation F1 (fake) ve ardından ROC-AUC kriterine göre en iyi C seçilir.
☐ Seçilen model test setinde yalnızca bir kez nihai değerlendirmeye alınır.

18. Deney E2 - HOG + LBP + KAZE ile RBF-SVM
İkinci deney birinci deneye KAZE'nin 128 boyutlu lokal anahtar-nokta bilgisini ekler. Toplam feature boyutu 6238 olur. SVM pipeline'ı ve C taraması E1 ile aynıdır. Bu deneyin temel karşılaştırma amacı, KAZE bilgisinin HOG+LBP tabanına eklenmesinin sınıflandırma performansına etkisini ölçmektir.
19. Deney E3 - HOG + LBP + KAZE ile Random Forest
Üçüncü deney aynı 6238 boyutlu fused feature vektörünü Random Forest sınıflandırıcısıyla değerlendirir. Böylece feature seti sabit tutulurken sınıflandırıcı ailesinin etkisi gözlemlenebilir.
Konfigürasyon n_estimators max_depth min_samples_lea
f
max_features
RF-1 300 None 1 sqrt
RF-2 300 20 1 sqrt
RF-3 300 None 2 sqrt
Random Forest için class_weight='balanced', random_state=42 ve n_jobs=-1 kullanılır. En iyi konfigürasyon yine validation F1 (fake), ardından validation ROC-AUC sırasıyla seçilir.
20. Model Kaydetme ve Fresh-Load Doğrulaması
Seçilen üç model joblib ile checkpoints klasörüne kaydedilir. atomic_joblib_save fonksiyonu modeli önce geçici bir dosyaya yazar, bu dosyayı yeniden yükleyerek okunabildiğini doğrular ve sonrasında os.replace ile hedef dosyanın yerine atomik biçimde taşır.
Kaydedilen modeller daha sonra tekrar joblib.load ile yüklenir ve test tahminlerinin eğitim sonunda elde edilen tahminlerle birebir aynı olup olmadığı assert ile kontrol edilir. Bu kontrol geçtiğinde fresh_load_inference_test.json içine PASSED bilgisi yazılır. Böylece checkpoint dosyasının bozuk olmadığı ve yeniden açıldığında aynı davranışı ürettiği doğrulanır.
21. Metrik, Tahmin ve Classification Report Çıktıları
Her modelin validation tarama sonuçları ayrı CSV dosyalarına yazılır. Seçilen nihai modellerin test metrikleri final_test_metrics.csv dosyasında bir araya getirilir. Ayrıca her model için scikit-learn classification_report çıktısı CSV olarak saklanır.
predictions/test_predictions.csv dosyası test setindeki her örnek için gerçek etiketi, her modelin tahminini ve karar skorunu veya fake olasılığını tutar. Metadata alanları mevcutsa sample_id, source_video, frame_index, face_index ve output_sha256 gibi alanlar da tahmin tablosuna eklenir.
Kaş Bölgesi Deepfake Tespiti - Kod ve Metodoloji Raporu
22. Üretilen Görseller
Kod üç temel görsel üretir ve figures klasörüne 600 DPI çözünürlükte kaydeder:
☐ model_performance_comparison.png: Accuracy, precision, recall, F1, ROC-AUC ve specificity değerlerini üç model için yan yana karşılaştırır.
☐ confusion_matrices.png: E1, E2 ve E3 modellerinin TN, FP, FN, TP dağılımını görselleştirir.
☐ roc_curves.png: Üç modelin ROC eğrilerini ve AUC değerlerini random classifier referans çizgisiyle birlikte gösterir.
23. Deney Konfigürasyonu, Ortam Kaydı ve Manifest
Son hücre deneyin tekrarlanabilirliğini ve izlenebilirliğini artıran bir dizi kayıt üretir.
Dosya İçerik / Amaç
config_resolved.yaml Veri yolu, feature parametreleri, seçilen model ayarları ve audit durumları
environment.json Python, platform, OpenCV, NumPy, pandas, scikit-learn ve scikit-image sürümleri
requirements_lock.txt pip freeze çıktısı; paket sürümlerinin tam kaydı
audit_summary.txt Veri sayıları, model seçim protokolü,

hash/video leakage durumu ve bütünlük
kontrolleri
run_summary.json Çalıştırma kimliği, süre, seçilen modeller ve
test metrikleri
output_manifest.csv Üretilen tüm dosyaların göreli yolu, boyutu ve
SHA-256 özeti
24. Kodun Güvenilirlik ve Tekrarlanabilirlik Önlemleri
☐ SEED=42 ile temel rastgelelik sabitlenir.
☐ OpenCV sürümü KAZE uyumluluğu için sabitlenir ve API çalışabilirliği başta doğrulanır.
☐ Veri klasörlerinin varlığı ve her splitte iki sınıfın bulunması kontrol edilir.
☐ Duplicate görüntü yolları reddedilir.
☐ SHA-256 tabanlı cross-split duplicate kontrolü yapılabilir.
☐ source_video varsa video-level leakage kontrolü uygulanır.
☐ Feature boyutları assert ile doğrulanır.
☐ NaN ve Inf featurelar kabul edilmez.
☐ Chunk cache'i veri yolu sırasını doğrulamadan yeniden kullanılmaz.
☐ Model seçimi validation setinde yapılır; test seçim için kullanılmaz.
☐ Checkpointler yeniden yüklenerek tahmin eşitliği doğrulanır.
☐ Her çıktı için SHA-256 manifesti üretilir.

25. Kodun Güçlü Yönleri
☐ Hazır kaş ROI veri setini değiştirmeden çalışır.
☐ Feature çıkarımı ile model eğitimini açık ve modüler fonksiyonlara ayırır.
☐ HOG, LBP ve KAZE'nin katkısını kontrollü deneylerle karşılaştırmaya izin verir.
☐ SVM ve Random Forest gibi iki farklı klasik sınıflandırıcı ailesini aynı feature seti üzerinde
test eder.
☐ Validation-test ayrımına uygun model seçim protokolü uygular.
☐ Chunk/cache mekanizması uzun feature çıkarımlarında Colab kesintilerine dayanıklılık
sağlar.
☐ Deney sonunda model, metrik, tahmin, grafik, environment ve manifest dosyalarını standart
klasör yapısında üretir.
26. Kodun Sınırlılıkları ve Dikkat Edilmesi Gereken Noktalar
☐ source_video alanı dolu değilse video-level split izolasyonu kod tarafından bağımsız
doğrulanamaz.
☐ KAZE descriptorları mean+std ile özetlendiğinden keypointlerin uzamsal düzeninin bir
bölümü kaybolur.
☐ SVM için yalnızca üç C değeri denenmektedir; daha geniş hiperparametre taraması
yapılmamaktadır.
☐ RBF-SVM'de gamma yalnızca 'scale' olarak sabittir.
☐ Random Forest için yalnızca üç önceden belirlenmiş konfigürasyon karşılaştırılır.
☐ Veri artırma (augmentation) uygulanmaz; kod yalnızca mevcut hazır ROI'leri kullanır.
☐ Model frame/ROI seviyesinde sınıflandırma yapar; video-level agregasyon bu notebookun
kapsamı değildir.
☐ Klasik handcrafted feature yaklaşımı, daha karmaşık deepfake artefaktlarını her zaman
yeterince temsil etmeyebilir.
27. Hücre Bazlı Kod Haritası
Hücre Ana İşlev Ürettiği / Etkilediği Çıktı
0 Notebook amacı, veri yapısı
ve deney tanımı Dokümantasyon
1 OpenCV 4.13 ve bağımlılık
kurulumu, KAZE doğrulaması Çalışma ortamı
2 local source bağlama, dizinler,
RUN_ID ve logging
Standart klasör yapısı,
training.log
3 ROI tarama, etiketleme,
metadata merge dataset_index.csv
4 Hash ve source_video leakage
audit split_audit.json
5 Ön işleme ve HOG/LBP/KAZE Feature fonksiyonları

fonksiyonları
6 Tüm örneklerden chunk
tabanlı feature çıkarımı
feature_chunks,
features_full.npz,
feature_metadata.csv
7
Split matrisleri, metrik

fonksiyonları ve güvenli
model kaydı
data_accounting.json
8 E1 HOG+LBP RBF-SVM Validation tablosu ve E1 test
sonuçları
9 E2 HOG+LBP+KAZE RBF-SVM Validation tablosu ve E2 test
sonuçları
10 E3 HOG+LBP+KAZE Random
Forest
Validation tablosu ve E3 test
sonuçları
11 Model/checkpoint, metrik,
tahmin ve figür kaydı
checkpoints, metrics,
predictions, figures
12 Config, environment, audit,
summary ve manifest
YAML/JSON/TXT/CSV deney
kayıtları
13 Çalıştırma ve resume notları Kullanım talimatı
28. Basitleştirilmiş Pseudocode
1. OpenCV/KAZE ortamını hazırla
2. local source package'ı bağla ve giriş/çıkış klasörlerini oluştur
3. real/fake × train/val/test görüntülerini tara
4. metadata.csv varsa görüntü indeksiyle birleştir
5. Split leakage denetimlerini gerçekleştir
6. Her ROI için:
    - 224×224 resize
    - grayscale
    - HOG çıkar (6084)
    - LBP çıkar (26)
    - KAZE çıkar (128)
7. HOG+LBP ve HOG+LBP+KAZE feature matrislerini oluştur
8. Mevcut train/val/test splitlerini ayır
9. E1: HOG+LBP → RBF-SVM, C ∈ {0.1,1,10}
10. E2: HOG+LBP+KAZE → RBF-SVM, C ∈ {0.1,1,10}
11. E3: HOG+LBP+KAZE → 3 Random Forest konfigürasyonu
12. Validation F1(fake), sonra ROC-AUC ile en iyi modeli seç
13. Seçilen modelleri bağımsız test setinde değerlendir
14. Modelleri joblib ile kaydet ve yeniden yükleme testi yap
15. Metrik, tahmin, classification report ve görselleri kaydet
16. Config, environment, audit, run summary ve manifest üret
29. Sonuç
Bu notebook, kaş bölgesine önceden kırpılmış görüntüler üzerinde klasik görüntü işleme özellikleri ve makine öğrenmesi sınıflandırıcılarını bir araya getiren, deneysel olarak karşılaştırılabilir ve çıktı yönetimi açısından izlenebilir bir deepfake tespit pipeline'ı kurmaktadır. Kodun temel metodolojik gücü, feature seti ile sınıflandırıcı etkisini ayrı deneyler halinde test etmesi ve model seçimini validation setiyle sınırlamasıdır. HOG şekil/yönelim

bilgisini, LBP lokal doku bilgisini, KAZE ise lokal anahtar-nokta descriptor bilgisini temsil eder. Bu üç feature ailesinin birleşimi RBF-SVM ve Random Forest üzerinde değerlendirilerek kaş ROI'lerinin real/fake ayrımındaki temsil gücü ölçülmektedir.
Ayrıca notebook yalnızca model eğitmekle kalmayıp veri bütünlüğü, feature bütünlüğü, checkpoint güvenilirliği, ortam sürümleri ve çıktı hashleri gibi deney tekrarlanabilirliğini destekleyen kontrolleri de içermektedir. Bu nedenle kod, akademik deney kaydı ve ekip içi model karşılaştırması için standartlaştırılmış bir çalışma pipeline'ı niteliğindedir.